

Smart Health Prescription System — Full Implementation Plan

This document describes the complete logic for a meal + workout + water + sleep recommendation system. Dataset used: V. Batthula, "Indian Food Nutrition," Kaggle. The system supports posture detection and live exercise feedback. Hand this document to any AI to implement the full logic correctly.

1. User Profile Inputs (Required)

Before generating any prescription, collect these from the user:

Field	Type	Values
age	integer	any (used for age modifier)
goal	enum	weight_loss, maintain, weight_gain
level	enum	beginner, intermediate, advanced
weight_kg	float	current body weight
height_cm	float	for BMI reference
gender	enum	male, female, other
daily_water_ml	integer	how much water user drank yesterday
sleep_hours	float	how many hours user slept last night

Water and sleep are collected daily (not one-time). They affect the **next week's prescription** through an adaptive modifier system described in Section 6.

2. Workout Prescription Logic

2.1 Base Matrix — Goal × Level

This is the core prescription before any modifiers are applied.

Goal	Level	Exercises/day	Sets/exercise	Reps/set	Rest between sets
weight_loss	beginner	5-6	3	12-15	30-45s
weight_loss	intermediate	6-7	3-4	12-15	30-45s
weight_loss	advanced	7-8	4	15-20	20-30s
maintain	beginner	4-5	3	10-12	60s
maintain	intermediate	5-6	3	10-12	60s
maintain	advanced	6-7	3-4	10-12	45-60s
weight_gain	beginner	5-6	3-4	8-10	60-90s
weight_gain	intermediate	6-8	4	6-10	60-90s
weight_gain	advanced	8-10	4-5	4-8	90-120s

Total sets formula: $\text{total_sets} = \text{exercises} \times \text{sets_per_exercise}$

- Beginner: ~18 sets/day
- Intermediate: ~24-32 sets/day
- Advanced: ~32-50 sets/day

2.2 Rep Range Logic by Goal

weight_loss	→ high reps, low rest (metabolic stress, calorie burn)
maintain	→ moderate reps, moderate rest (muscle preservation)
weight_gain	→ low reps, heavy weight, long rest (hypertrophy / strength)

2.3 Exercise Type Priority by Goal

weight_loss:	→ compound movements + cardio circuits (burpees, squats, lunges, mountain climbers)
maintain:	→ balanced push/pull/legs split, moderate intensity
weight_gain:	→ heavy compound lifts (bench press, deadlift, squat, overhead press, rows)

2.4 Muscle Group Rotation (Day Split)

Rotate exercises daily to avoid overtraining same muscle groups:

```
Day 1: Push (chest, shoulders, triceps)
Day 2: Pull (back, biceps)
Day 3: Legs (quads, hamstrings, glutes, calves)
Day 4: Core + Cardio
Day 5: Full body (beginner) OR repeat Day 1 cycle (intermediate/advanced)
Day 6: Active recovery (light stretching, walking)
Day 7: Rest
```

3. Age Modifier (Applied AFTER base matrix)

Age is a post-processing modifier. Do not rebuild the base matrix for age — apply these adjustments on top of the base prescription.

```
python
```

```
def apply_age_modifier(prescription, age):
    if age < 18:
        prescription['sets'] = max(2, prescription['sets'] - 1)
        prescription['reps_min'] = 12
        prescription['reps_max'] = 15
        prescription['rest_seconds'] += 15
        prescription['meal_note'] = "No supplements. Add 1 milk/fruit item per

    elif 18 <= age <= 35:
        pass # Baseline group – no change needed

    elif 36 <= age <= 50:
        prescription['rest_seconds'] = int(prescription['rest_seconds'] * 1.2)
        prescription['exercises'] = min(prescription['exercises'], 8)
        prescription['reps_min'] = max(prescription['reps_min'], 8)
        prescription['meal_note'] = "Swap 1 carb item for fibre-rich sabzi or d

    elif 51 <= age <= 60:
        prescription['exercises'] = max(4, prescription['exercises'] - 2)
        prescription['rest_seconds'] = max(60, prescription['rest_seconds'])
        prescription['avoid'] = ["high_impact", "heavy_jumps", "running"]
        prescription['substitute'] = "Replace high-impact with walking/cycling/"
        prescription['meal_note'] = "Add 1 extra protein item per meal slot."

    else: # 60+
        prescription['exercises'] = min(prescription['exercises'], 6)
        prescription['sets'] = min(3, prescription['sets'])
        prescription['reps_min'] = 10
        prescription['reps_max'] = 12
        prescription['rest_seconds'] = max(90, prescription['rest_seconds'])
        prescription['level_cap'] = 'beginner' # Force beginner regardless of
        prescription['meal_note'] = "Prioritise calcium and protein-rich items.

    return prescription
```

4.1 Meal Items per Slot by Goal

Meal Slot	Weight Loss	Maintain	Weight Gain	Timing
Breakfast	3-4 items	4-5 items	5-6 items	7-9 AM
Lunch	4-5 items	5-6 items	6-7 items	12-2 PM
Snacks	1-2 items	2-3 items	3-4 items	4-6 PM
Dinner	3-4 items	4-5 items	5-6 items	7-9 PM

Total items per day:

- Weight loss: 11-15 items
- Maintain: 15-19 items
- Weight gain: 19-23 items

4.2 Calorie Distribution by Meal Slot

Breakfast: 25-30% of daily calories
Lunch: 35-40% of daily calories
Snacks: 10-15% of daily calories
Dinner: 20-25% of daily calories

Daily calorie targets (approximate):

weight_loss: TDEE - 300 to 500 kcal
maintain: TDEE (maintenance calories)
weight_gain: TDEE + 300 to 500 kcal

4.3 Item Selection Logic from Indian Food Nutrition Dataset

Filter and rank items from the dataset using this priority:

python

```
def select_meal_items(dataset, goal, meal_slot, count):
    # Step 1: Filter by calorie density
    if goal == 'weight_loss':
        filtered = dataset[dataset['calories_per_100g'] < 200]
    elif goal == 'maintain':
        filtered = dataset[dataset['calories_per_100g'].between(150, 400)]
    else: # weight_gain
        filtered = dataset[dataset['calories_per_100g'] > 250]

    # Step 2: Sort by protein-to-calorie ratio (all goals prefer high protein)
    filtered['protein_ratio'] = filtered['protein_g'] / filtered['calories']
    filtered = filtered.sort_values('protein_ratio', ascending=False)

    # Step 3: Apply meal slot preference
    slot_preferences = {
        'breakfast': ['light', 'medium'],
        'lunch':      ['medium', 'heavy'],
        'snacks':     ['light'],
        'dinner':     ['light', 'medium']
    }

    # Step 4: Return top N items, rotating to avoid repetition
    return filtered.head(count * 3).sample(count) # sample from top pool for v
```

4.4 Indian Food Category → Meal Slot Mapping

Meal Slot	Weight Loss	Maintain	Weight Gain
Breakfast	Idli, Poha, Upma, Sprouts	Dosa, Egg bhurji, Paratha (1), Chai	Paratha (2+), Paneer bhurji, Banana, Full-fat milk
Lunch	Moong dal, Grilled sabzi, Brown rice (small)	Dal, Roti (2), Sabzi, Rice (medium)	Dal makhani, Rice (large), 2 Roti, Sabzi, Paneer, Raita
Snacks	Fruit (1), Buttermilk	Roasted chana, Fruit, Chai	Banana shake, Chana chaat, Peanut chikki, Curd
Dinner	Dal soup, Grilled paneer, Salad	Roti (2), Dal, Sabzi, Raita	Rice + Dal + Sabzi + Paneer + Roti

4.5 Variety Rule

Maintain a 7-day rotation pool. Never repeat the same item more than twice in 7 days.

```
python

# Pre-filter a pool of 15-20 items per goal from the dataset
# On each day, sample from this pool without replacement (within the week)
# Reset the pool every 7 days
```

5. Water & Sleep — Daily Collection and Feedback

5.1 Water Intake Targets

Goal	Baseline Target
weight_loss	3.0–3.5 litres/day
maintain	2.5–3.0 litres/day
weight_gain	3.0–3.5 litres/day

Age modifier:

```
< 18:  reduce target by 0.3L
> 60:  reduce target by 0.3L
36-60: increase target by 0.2L (recovery support)
```

5.2 Sleep Targets

Level	Minimum Sleep	Optimal Sleep
Beginner	7 hours	8 hours
Intermediate	7.5 hours	8-9 hours
Advanced	8 hours	8-9 hours

6. Adaptive Weekly Modifier (The "Crazy Smart" Part)

This is the core adaptive intelligence layer. Water and sleep logged daily are aggregated at the end of each week. The system then adjusts the prescription for the following week automatically.

6.1 How It Works

At the end of each week (7 days), compute:

```
avg_sleep = mean of all 7 daily sleep_hours values
avg_water = mean of all 7 daily water_ml values
sleep_target = target from Section 5.2
water_target = target from Section 5.1
```

Then apply the weekly adaptive modifier:

6.2 Sleep Deficit Modifier

```
python
```



```

def sleep_modifier(avg_sleep, sleep_target, prescription):
    deficit = sleep_target - avg_sleep

    if deficit >= 2.0: # Severe: slept 2+ hours less than target
        prescription['exercises'] = max(3, prescription['exercises'] - 2)
        prescription['sets'] = max(2, prescription['sets'] - 1)
        prescription['rest_seconds'] += 30
        prescription['intensity'] = 'reduced'
        prescription['system_message'] = (
            "Your sleep was significantly below target this week. "
            "Workout intensity has been reduced to support recovery. "
            "Focus on sleeping 30 minutes earlier each night next week."
        )

    elif 1.0 <= deficit < 2.0: # Moderate: slept 1-2 hours less
        prescription['exercises'] = max(4, prescription['exercises'] - 1)
        prescription['rest_seconds'] += 15
        prescription['intensity'] = 'slightly_reduced'
        prescription['system_message'] = (
            "Mild sleep deficit detected. One exercise removed from your plan. "
            "Aim for 30 more minutes of sleep each night."
        )

    elif deficit <= -0.5: # Surplus: slept more than target
        prescription['rest_seconds'] = max(20, prescription['rest_seconds'] - 1)
        prescription['system_message'] = (
            "Great sleep this week! Recovery is optimised. "
            "Rest time slightly reduced to increase workout efficiency."
        )

    return prescription

```

6.3 Water Deficit Modifier

python

```

def water_modifier(avg_water_ml, water_target_ml, prescription):
    deficit_ratio = (water_target_ml - avg_water_ml) / water_target_ml

    if deficit_ratio >= 0.3: # Drank 30%+ less than target
        prescription['exercises'] = max(3, prescription['exercises'] - 1)
        prescription['cardio_intensity'] = 'low'
        prescription['meal_items']['snacks'] += 1 # Add an extra hydrating food
        prescription['system_message'] = (
            "You were significantly under-hydrated this week. "
            "High-intensity cardio has been reduced. "
            "Added a hydrating food item (like coconut water or cucumber raita)"
        )

    elif 0.1 <= deficit_ratio < 0.3: # Mild deficit
        prescription['system_message'] = (
            "You were slightly under your water target this week. "
            "Try carrying a 1L bottle and refilling it 3 times daily."
        )

    elif deficit_ratio <= -0.1: # Over-hydrated (rare but track it)
        prescription['system_message'] = "Excellent hydration this week! Keep it up!"

    return prescription

```

6.4 Combined Weekly Modifier — Full Function

python

```
def apply_weekly_adaptive_modifier(base_prescription, weekly_logs):
    """
    weekly_logs: list of 7 dicts, each with keys:
        - sleep_hours (float)
        - water_ml (int)
    """
    avg_sleep = sum(d['sleep_hours'] for d in weekly_logs) / 7
    avg_water = sum(d['water_ml'] for d in weekly_logs) / 7

    prescription = base_prescription.copy()
    prescription = sleep_modifier(avg_sleep, prescription['sleep_target'], pres
    prescription = water_modifier(avg_water, prescription['water_target'], pres

    # Log the adaptation reason for transparency to user
    prescription['adaptation_log'] = {
        'week_avg_sleep': round(avg_sleep, 1),
        'week_avg_water_ml': int(avg_water),
        'sleep_vs_target': round(avg_sleep - prescription['sleep_target'], 1),
        'water_vs_target_ml': int(avg_water - prescription['water_target']),
        'changes_applied': prescription.get('system_message', 'No changes neede
    }

    return prescription
```

7. Posture Detection Integration

The posture detection module feeds into the workout prescription as a form-quality score.

7.1 Form Score

After each workout session, compute:

```
form_score = correct_reps / total_reps (value between 0.0 and 1.0)
```

7.2 Form Modifier (Applied Next Session)

```
python
```

```
def apply_form_modifier(prescription, form_score):
    if form_score < 0.5: # Poor form
        prescription['reps_max'] = prescription['reps_min'] # Reduce rep range
        prescription['weight_suggestion'] = 'reduce_by_10_percent'
        prescription['system_message'] = (
            "Your form needs improvement. Reduce weight and focus on controlled
            "Speed has been slowed – aim for 3 seconds down, 1 second up."
        )

    elif 0.5 <= form_score < 0.75: # Acceptable form
        prescription['system_message'] = (
            "Form is acceptable. Focus on the highlighted joints in the feedback"
        )

    elif form_score >= 0.9: # Excellent form – allow progression
        prescription['weight_suggestion'] = 'increase_by_5_percent'
        prescription['system_message'] = (
            "Excellent form this session! You're ready to increase weight slight"
        )

    return prescription
```

8. Full Prescription Generation — Master Function

python

```

def generate_full_prescription(user_profile, weekly_logs=None, last_form_score=None):
    """
    user_profile keys:
        age, goal, level, weight_kg, height_cm, gender,
        daily_water_ml (today), sleep_hours (last night)

    weekly_logs: list of 7 daily dicts (sleep_hours, water_ml) – used from Week 2 onward
    last_form_score: float 0.0-1.0 from posture detection module
    """

    # Step 1: Base workout prescription
    prescription = get_base_prescription(user_profile['goal'], user_profile['level'])

    # Step 2: Apply age modifier
    prescription = apply_age_modifier(prescription, user_profile['age'])

    # Step 3: Apply weekly adaptive modifier (from Week 2 onward)
    if weekly_logs and len(weekly_logs) == 7:
        prescription = apply_weekly_adaptive_modifier(prescription, weekly_logs)

    # Step 4: Apply posture/form modifier (from Session 2 onward)
    if last_form_score is not None:
        prescription = apply_form_modifier(prescription, last_form_score)

    # Step 5: Generate meal plan
    meal_plan = generate_meal_plan(
        goal=user_profile['goal'],
        age=user_profile['age'],
        weight_kg=user_profile['weight_kg']
    )

    # Step 6: Set water and sleep targets
    prescription['water_target'] = get_water_target(user_profile['goal'], user_profile['level'])
    prescription['sleep_target'] = get_sleep_target(user_profile['level'])

    return {
        'workout': prescription,
        'meals': meal_plan,
        'targets': {
            'water_ml': prescription['water_target'],
            'sleep_hours': prescription['sleep_target']
        },
    }

```

```
    'adaptation_log': prescription.get('adaptation_log', None)
}
```

9. Summary of All Modifier Priority Order

When multiple modifiers apply, apply them in this exact order to avoid conflicts:

- | | |
|-------------------------------|------------------------|
| 1. Base matrix (goal × level) | ← always first |
| 2. Age modifier | ← demographic baseline |
| 3. Weekly sleep modifier | ← recovery state |
| 4. Weekly water modifier | ← hydration state |
| 5. Posture/form modifier | ← performance feedback |

Each modifier only reduces or adjusts — it never increases beyond the base matrix values. This prevents over-prescription.

10. Data Storage Schema (Recommended)

```
python
```

```

# User profile (stored once, updated if user changes goal/level)
user_profile = {
    "user_id": "str",
    "age": int,
    "goal": "weight_loss | maintain | weight_gain",
    "level": "beginner | intermediate | advanced",
    "weight_kg": float,
    "height_cm": float,
    "gender": "str"
}

# Daily log (one entry per day)
daily_log = {
    "user_id": "str",
    "date": "YYYY-MM-DD",
    "water_ml": int,
    "sleep_hours": float,
    "form_score": float,      # from posture detection, 0.0-1.0
    "workout_completed": bool
}

# Weekly prescription (generated every Monday)
weekly_prescription = {
    "user_id": "str",
    "week_start": "YYYY-MM-DD",
    "workout": { ... },      # full workout object
    "meals": { ... },        # full meal plan object
    "targets": { ... },      # water + sleep targets
    "adaptation_log": { ... } # why changes were made this week
}

```

11. Week 1 Behaviour (Cold Start)

On Week 1, no historical data exists. The system should:

- Use base matrix + age modifier only (no weekly adaptive modifier)
- Set form_score = None (no posture data yet)
- Start collecting daily water + sleep logs from Day 1
- On Day 8 (start of Week 2), run the weekly adaptive modifier for the first time
- Show the user a message: "Your plan will personalise further after your first week based on your sleep and hydration."

End of plan. All logic above is self-contained and can be implemented in Python, Node.js, or any backend language. The Indian Food Nutrition dataset (Batthula, Kaggle) is used for meal item selection in Section 4.